MANTA: Multiview Anomaly Detection for Tiny object

A report submitted for the course named

Project – II (CS-4202)

Submitted By

Pratham Gupta

Semester VII

230103034

CSE (Artificial Intelligence and Data Science)

Supervised By

Dr. N. Kishorjit Singh

Assistant Professor,

Department of Computer Science and Engineering



Department of Computer Science and Engineering

Indian Institute of Information Technology Senapati, Manipur

November 2026

Declaration

I declare that this report presents the project work in my own words. Material drawn from published sources is acknowledged in the references. The results reported here are taken from the saved execution output of the accompanying MANTA notebook; no unrecorded experiments or measurements are presented as part of this report.

Name: Pratham Gupta	Roll number: 230103034
Signature: _____	Date: _____

Department of Computer Science and Engineering
Indian Institute of Information Technology Senapati, Manipur

Dr. N. Kishorjit Singh
Registrar, IIIT Manipur

Email: kishorjit@iiitmanipur.ac.in
Contact No.: +91 8974008610

To Whom It May Concern

This is to certify that the project report entitled **"MANTA: Multiview Anomaly Detection for Tiny object"**, submitted by **Pratham Gupta** (roll number **230103034**) for the course **CS-4202**, is presented for academic evaluation in the Department of Computer Science and Engineering, Indian Institute of Information Technology Senapati, Manipur.

Signature of the Supervisor
Dr. N. Kishorjit Singh

Signature of Examiner 1: .....

Signature of Examiner 2: .....

Signature of Examiner 3: .....

Signature of Examiner 4: .....

Abstract

This report presents MANTA, a supervised image-level anomaly detection pipeline for maize, paddy, soybean, and wheat. Its task is to distinguish normal from anomalous images within each crop category. Agricultural images can vary in texture, illumination, orientation, and defect appearance, making anomaly recognition challenging. The project addresses this task by combining pretrained visual representations with spatial feature processing, and documents the implementation and results recorded in the supplied notebook.

The pipeline uses a frozen DINOv2-Large vision transformer to extract patch features from four intermediate depths. These features are concatenated and projected to a shared dimension, preserving the 16-by-16 patch layout. A spatial graph transformer then refines patch representations using a symmetric neighborhood mask. The model pools both the projected and graph-refined features, combines them through a learned gate, and applies cross-attention before a binary classification head generates normal/anomaly scores. Training combines classification, supervised contrastive, graph-smoothness, and reconstruction objectives. Cross-attention maps are retained for qualitative inspection of image regions influencing the model.

Each crop category is evaluated separately using the notebook's seeded, stratified split. The recorded image-level AUROC (I-AUROC) values are 99.00% for maize, 99.25% for paddy, 99.39% for soybean, and 98.22% for wheat, with a reported mean of 98.96%. These values describe the saved run on one split formed from category folders named `test`. Since no independent external evaluation or repeated split is reported, the results should be interpreted as performance on this experimental partition rather than evidence of generalization to unseen collections.

Acknowledgement

I would like to express my sincere gratitude to Dr. N. Kishorjit Singh, my project supervisor, for his guidance, constructive feedback, and encouragement during this project. His advice helped me clarify the project objectives, organize the system design, and communicate the implementation and findings more effectively.

I am thankful to the faculty and staff of the Department of Computer Science and Engineering at the Indian Institute of Information Technology Senapati, Manipur, for providing an academic environment and institutional resources that supported this work. I also acknowledge the authors and contributors of DINOv2 and the developers of the open-source software libraries used in the project; their research, tools, and documentation supported the model implementation and evaluation.

Finally, I thank my classmates and peers for their discussions and feedback, and my family for their patience and encouragement throughout the completion of this report.

Contents

Declaration i

To Whom It May Concern ii

Abstract iii

Acknowledgement iv

1 Introduction 1

1.1 Background and Motivation 1
1.2 Project Context 1
1.3 Problem Statement 2
1.4 Aim and Objectives 2
1.5 Research Questions 2
1.6 Scope and Delimitations 3
1.7 Contributions of the Work 3
1.8 Organization of the Report 3

2 Literature Survey 4

2.1 Visual Anomaly Detection in Context 4
2.2 Vision Transformers 5
2.3 Self-Supervised Visual Pretraining and DINOv2 5
2.4 Graph Representation of Patch Relationships 5
2.5 Supervised Contrastive Learning 6
2.6 Feature Fusion and Cross-Attention 6
2.7 Hybrid Training Objectives 6
2.8 AUROC as an Image-Level Metric 7
2.9 Positioning of MANTA 7
2.10 Literature-to-Implementation Summary 7

3 Requirement Engineering **9**

3.1 Stakeholders and Intended Use . . . 9

3.2 Functional Requirements . . . 9

3.3 Non-Functional Requirements . . . 10

3.4 Input Data Requirements . . . 11

3.5 Operating and Environment Requirements . . . 11

3.6 Use Cases . . . 11

3.6.1 Train Category Model . . . 11

3.6.2 Score an Image . . . 11

3.6.3 Review Attention Overlay . . . 12

3.7 Constraints, Assumptions, and Risks . . . 12

3.8 Acceptance Criteria . . . 12

4 System Design **13**

4.1 Architectural Overview . . . 13

4.2 Subsystem Decomposition . . . 14

4.2.1 Dataset Discovery . . . 14

4.2.2 Image Dataset and Transform . . . 14

4.2.3 Frozen Feature Backbone . . . 15

4.2.4 Multi-Depth Projection . . . 15

4.2.5 Spatial Graph Transformer . . . 15

4.2.6 Illustrative Patch Neighborhood . . . 15

4.2.7 Knowledge Fusion . . . 16

4.2.8 Cross-Attention and Classifier . . . 16

4.3 Feature and Graph Notation . . . 16

4.4 Graph Transformer Layer . . . 16

4.5 Gated Fusion and Cross-Attention Equations . . . 17

4.6 Hybrid Loss . . . 17

4.6.1 Classification Loss . . . 17

4.6.2 Supervised Contrastive Loss . . . 17

4.6.3 Graph Smoothness Loss . . . 18

4.6.4 Knowledge Reconstruction Loss . . . 18

4.7 Training and Evaluation Flows . . . 18

4.8 Algorithmic Outline . . . 18

4.9 Complexity and Resource Considerations . . . 19

5 Implementation **20**

5.1 Notebook Organization . . . 20
5.2 Configuration Management . . . 20
5.3 Category Discovery and Binary Labels . . . 20
5.4 Image Transformation . . . 21
5.5 Backbone Loading and Feature Use . . . 21
5.6 Model Components . . . 21
5.7 Optimization Loop . . . 22
5.8 Evaluation and Export . . . 22
5.9 Outputs Produced . . . 22
5.10 Error Handling and Operational Notes . . . 22
5.11 Reproducibility Record . . . 23

6 Results and Discussion **24**
6.1 Evaluation Protocol in the Saved Run . . . 24
6.2 Category-Level Scores . . . 24
6.3 Per-Category Discussion . . . 25
6.3.1 Maize . . . 25
6.3.2 Paddy . . . 25
6.3.3 Soybean . . . 25
6.3.4 Wheat . . . 25
6.4 Training Loss Trace . . . 25
6.5 Interpretation of AUROC . . . 26
6.6 Confusion Matrix, Accuracy, and F1-Score . . . 26
6.6.1 Threshold and Metric Reporting . . . 27
6.6.2 Recalculation After a Notebook Rerun . . . 28
6.7 Attention Maps and Pixel-Level Evidence . . . 29
6.7.1 Saved Cross-Attention Figures . . . 29
6.8 Published MANTA Benchmark Results . . . 31
6.8.1 Dataset Profile . . . 31
6.8.2 Single-View Results . . . 31
6.8.3 Multi-View Results . . . 31
6.8.4 Multi-Class Results . . . 32
6.8.5 Text-Prompt Results . . . 32
6.8.6 Visual-Language Results . . . 32
6.8.7 Qualitative Findings . . . 33
6.9 Threats to Validity . . . 33

6.9.1 Data Split . . . 33
6.9.2 Sample and Category Coverage . . . 33
6.9.3 Model Selection and Reproducibility . . . 33
6.9.4 Metric Coverage . . . 33
6.10 Summary of Findings . . . 33

7 Conclusion and Future Work **35**
7.1 Conclusion . . . 35
7.2 Recommended Next Steps . . . 35
7.3 Final Remarks . . . 36

A Run Configuration and Dataset Inventory **37**
A.1 Model and Training Configuration . . . 37
A.2 Categories and Defect Groups . . . 38
A.3 Partition Counts . . . 38
A.4 Training Checkpoint Values . . . 39
A.5 Expected Folder Layout . . . 39

B Algorithms, Verification, and Reporting Checklist **40**
B.1 Category Training Pseudocode . . . 40
B.2 Suggested Verification Cases . . . 41
B.3 Rerun Checklist . . . 42
B.4 Result Reporting Template . . . 42

C Glossary and Notation **44**
Notation . . . 45

References **46**

List of Figures

4.1 MANTA architecture with a raw-feature bypass into gated fusion, graph-masked refinement, and a single reported image-level I-AUROC result. 14

4.2 Illustrative 8-neighbor patch neighborhood. The center token attends to its selected nearby tokens and itself; the full 16×16 graph mask is symmetrized. . . . 15

6.1 Illustrative pooled confusion-matrix example from the supplied reference. It is not a measured result from the saved notebook. 27

6.2 Maize: six normal-class input composites and their cross-attention overlays. . . . 29

6.3 Paddy: six normal-class input composites and their cross-attention overlays. . . . 29

6.4 Soybean: six normal-class input composites and their cross-attention overlays. . . 29

6.5 Wheat: six normal-class input composites and their cross-attention overlays. . . . 30

List of Tables

2.1 Relationship between background concepts and notebook components. 8

3.1 Functional requirements and implementation status. 9
3.2 Non-functional requirements and current status. 10
3.3 Project risks and practical mitigations. 12

6.1 Saved category-wise image-level results. 24
6.2 Mean total training loss at the logged epochs. 25
6.3 Metrics calculated from the illustrative pooled confusion matrix in Table 6.4. Values other than the confusion counts are derived from that example, not measured notebook predictions. 28
6.4 Illustrative pooled confusion counts used to calculate the metrics in Table 6.3. . . 28
6.5 Published MANTA benchmark profile.[9] 31
6.6 Single-view results across five domains (I-AUROC/P-AUROC, %).[9] 31
6.7 Published multi-view MANTA results by domain (I-AUROC/P-AUROC, %).[9] . 31
6.8 Published MANTA benchmark comparisons and interpretation.[9] 32
6.9 Published multi-class MANTA averages (%).[9] 32
6.10 Published one-shot text-prompt evaluation summary on a balanced 1,000-image test set.[9] 32
6.11 Published average visual-language accuracy (mean ± reported standard deviation, %). The ± values denote variability, not additive gains.[9] 32
6.12 Published qualitative observations and practical implications.[9] 33

A.1 Configuration values used in the saved MANTA run. 37
A.2 Defect-group directory names found in the saved run. 38
A.3 Aggregate label counts from category discovery. 38
A.4 Training loss checkpoints reproduced from the notebook log. 39

B.1 Verification cases for a reproducible rerun. 41
B.2 Recommended fields for future category-level results. 43

C.1 Terms used in this report. 44

Chapter 1

Introduction

1.1 Background and Motivation

An image-based inspection system has to do more than identify a familiar object. In many practical settings it must decide whether an observed item differs from an expected appearance, even when the exact variation is not known in advance. Agricultural imagery presents this problem in a particularly visible form: a crop image may show a healthy example, a single defect, or several kinds of damage together. The appearance of an abnormality can be small relative to the whole image, while texture, lighting, orientation, and background may vary between samples.

Manual review remains useful because an experienced reviewer can combine visual evidence with domain knowledge. It is nevertheless difficult to inspect a large collection consistently by eye. A computer-vision pipeline can support review by ranking images according to their likelihood of being anomalous and by presenting a visual cue about the image regions associated with its prediction. Such a pipeline should be viewed as a decision-support tool unless it has been validated for the setting in which it will be used.

The project implemented in the supplied notebook is called MANTA. It applies a supervised classifier to the agricultural categories maize, paddy, soybean, and wheat. Rather than learning all visual features from scratch on the notebook's relatively small per-category split, it uses a pre-trained DINOv2-Large visual backbone and keeps that backbone frozen. The trainable portion combines patch-level processing, graph-constrained attention, feature fusion, and a classification head.

1.2 Project Context

The notebook implements an end-to-end workflow rather than a single model class. It discovers the categories and their class folders, creates category-specific train and test partitions, transforms the images, constructs a spatial mask for patch interactions, trains one MANTA model per category, and saves a result table and example attention overlays. Because it evaluates each crop category independently, a maize classifier is not directly asked to classify a paddy image. The target task within a category is binary: normal versus anomalous.

The architecture replaces a prior PatchCore baseline referred to in the notebook comments [1]. The distinction matters for data preparation. A memory-bank anomaly detector can be trained from normal examples only, whereas the current architecture contains a trainable binary classification head. It therefore needs labeled normal and anomalous examples in its training

partition. The experiment is consequently a supervised anomaly-classification study, not a one-class detector trained solely on healthy images.

1.3 Problem Statement

Let x be an image from one of the four crop categories, and let $y \in \{0, 1\}$ denote its label, with $y = 0$ for normal and $y = 1$ for anomalous. For each category, the system learns a function

$$f_\theta(x) = p(y = 1 \mid x), \tag{1.1}$$

where the probability is produced by a category-specific classifier. The principal evaluation question is whether anomalous images receive higher scores than normal images in the held-out partition. The notebook measures this with image-level AUROC.

A second, related question is whether the model's internal patch attention can help a person inspect the prediction. The notebook reshapes the cross-attention weights into the patch grid, upsamples the map, and overlays it on the original image. This is a visualization of model attention, not a direct estimate of defect pixels. Without comparison to confirmed pixel-level ground-truth masks, the overlay cannot establish localization accuracy.

1.4 Aim and Objectives

The aim is to document and implement the MANTA supervised agricultural anomaly classifier. The objectives are:

1. Use a frozen DINOv2-Large backbone to extract patch features at several transformer depths.
2. Combine these feature depths into a fixed-dimensional patch representation.
3. Represent the image patch grid as a spatial graph and restrict attention according to a fixed neighbor mask.
4. Fuse unrefined and graph-refined image summaries with a learned gate.
5. Use cross-attention and binary classification to score images.
6. Train with a weighted objective containing classification, contrastive, graph-smoothness, and reconstruction losses.
7. Evaluate each category with the saved image-level AUROC output and document the limitations of the current test protocol.

1.5 Research Questions

The implementation and its recorded run are organized around the following practical questions:

- Can frozen pretrained visual representations support a supervised anomaly classifier on the four supplied crop categories?
- Does adding graph-restricted attention provide a coherent way to exchange information among neighboring patch tokens?

- What image-level AUROC values are recorded for each crop category under the notebook's seeded split?
- What additional evidence is required before interpreting attention overlays as defect localization or applying the classifier outside this split?

These questions are intentionally limited to the available implementation and execution log. The notebook does not include controlled ablations, repeated seeds, or an external test set, so it cannot answer whether each architectural component independently improves performance.

1.6 Scope and Delimitations

The project covers four categories and two image-level labels. All defect-type folders for a crop are combined into the single anomalous label during classification; defect subtype names are preserved for display but are not separate output classes. The image transform resizes inputs to 224×224, and DINOv2's patch size of 14 yields a 16×16 patch grid. The model is trained for 15 epochs per category using the parameters recorded in the notebook.

The report does not claim a diagnosis of a specific plant disease, a pixel-accurate segmentation, a calibrated risk score, or suitability for field deployment. No inference latency, throughput, memory benchmark, confidence interval, external dataset result, or human-review study is reported in the saved notebook output. The evaluation split is generated from the category folders named `test`; it is not described as an independently collected test cohort. These delimitations are carried through the results and conclusion.

1.7 Contributions of the Work

The work documented here has four concrete outputs. First, it provides a reproducible description of the notebook's multi-stage MANTA design, from pretrained feature extraction to binary scoring. Second, it records the actual category-wise split counts and image-level AUROC values rather than presenting only a qualitative claim. Third, it distinguishes executable pixel-metric code from pixel-metric results actually present in the saved notebook output. Fourth, it identifies a set of follow-up experiments that would strengthen generalization and localization evidence.

1.8 Organization of the Report

Chapter 2 reviews the model ideas relevant to the implementation. Chapter 3 describes requirements, input assumptions, operating constraints, and acceptance checks. Chapter 4 explains the system design and the model's mathematical components. Chapter 5 details data handling, training, evaluation, and outputs. Chapter 6 presents the saved results and discusses what they do and do not establish. Chapter 7 concludes the report and prioritizes future work. The appendices collect the configuration, category labels, procedural outline, test cases, and terminology for reproducibility.

Chapter 2

Literature Survey

2.1 Visual Anomaly Detection in Context

Anomaly detection asks whether an observation departs from an expected pattern. In image analysis, the term covers several related tasks. An image-level method returns one score for an entire image. A pixel-level method assigns a score to each location, often to identify the extent of a defect. A classification system trained with normal and anomalous labels is supervised, while a one-class method may use only normal training examples and treat departures from them as anomalous.

These distinctions affect how a method should be trained and evaluated. An image-level score can be assessed using image labels, while localization claims require pixel-level labels or another explicit reference. A model can rank anomalous images well while assigning attention to a broad region that does not match a defect mask. For that reason, the present report treats the MANTA classifier and its attention overlay as separate outputs with different evidence requirements.

Representative one-class approaches include PaDiM, which models patch-feature distributions, and PatchCore, which uses a memory bank of nominal patch features for nearest-neighbor anomaly scoring [2, 1]. Both provide context for normal-only training; by contrast, the MANTA implementation in this report uses labeled normal and anomalous images for binary classification. MVTec AD is a widely used real-world benchmark for unsupervised industrial anomaly detection with pixel-level annotations, and is distinct from the agricultural categories studied here [3].

Other anomaly-detection methods illustrate different training assumptions and scoring strategies. Deep SVDD learns a compact representation of normal data [13]; CutPaste and DRAEM use synthetic defects to construct training signals [14, 15]. Student-teacher feature comparison and FastFlow's normalizing-flow density estimation provide further alternatives for anomaly localization [16, 17]. Reverse distillation also compares student representations with a teacher-derived target [18]. These methods are relevant background, not baselines evaluated in the supplied MANTA run.

Benchmark coverage has also broadened beyond the original MVTec AD setting. MVTec LOCO AD includes logical and structural anomalies, while VisA provides a large-scale industrial visual-anomaly dataset with image- and pixel-level annotations [24, 25]. Their object domains and collection protocols differ from the crop categories used here, so scores across these datasets should not be treated as directly comparable.

2.2 Vision Transformers

The transformer architecture introduced self-attention as a mechanism for sequence modeling without recurrent processing [10]. The Vision Transformer (ViT) adapts this design to images by dividing an image into fixed-size patches, embedding each patch as a token, and processing the resulting sequence with transformer layers [4]. Since self-attention relates each token to other tokens, a representation can incorporate broader image context rather than relying only on local neighborhoods [4]. Thus, the patch grid serves both as a spatial decomposition of the image and as the sequence processed by attention.

The MANTA notebook uses this patch-based structure directly. At input size 224×224 and patch size 14, the spatial token grid is 16×16, giving 256 patch tokens. These are the nodes passed to the graph transformer. The patch grid enables a fixed neighborhood relation to be constructed from row and column coordinates, independently of image content.

2.3 Self-Supervised Visual Pretraining and DINOv2

A common challenge in a small labeled dataset is that training a large visual model from random initialization may require more examples than are available. Pretraining offers a way to reuse representations learned from a broader image collection. Self-distillation methods such as DINO learn visual features without manual labels [11], while iBOT combines masked image modeling with an online tokenizer [12]. DINOv2 extends this line of self-supervised visual feature learning toward general-purpose representations [5]. The supplied notebook loads the DINOv2-Large model, places it in evaluation mode, and disables gradient updates to its parameters.

Freezing the backbone changes the optimization problem. The model does not adapt the DINOv2 transformer weights to crop defects. Instead, a trainable projection and the subsequent graph, fusion, attention, and classification modules learn how to use the fixed representation for the target labels. This reduces the trainable part of the system and may help when the supervised dataset is limited, although this design does not guarantee resistance to domain shift.

The notebook asks the backbone for intermediate patch features from the last four transformer depths. These features have the same patch-grid geometry but different transformer depths. Calling them multi-depth features is more precise than claiming they are image pyramids at different spatial resolutions: the spatial grid remains 16×16 in this configuration.

2.4 Graph Representation of Patch Relationships

A graph describes entities as nodes and selected relationships as edges. For an image grid, a node can represent one patch and an edge can connect two nearby patch locations. The MANTA implementation builds a fixed spatial k-nearest-neighbor mask from Euclidean distances between grid coordinates. The mask includes self-connections and is symmetrized, so a connection selected in either direction is permitted in both directions.

Graph Convolutional Networks aggregate features over graph neighborhoods [20], while Graph Attention Networks compute learned attention over graph neighborhoods [6]. Graphormer further studies transformer architectures adapted to graph representation learning [21]. MANTA shares the neighborhood-aware attention motivation, but its implementation uses standard multi-head transformer attention with a dense adjacency mask rather than the original GAT or Graphormer formulations.

The graph transformer uses multi-head attention with the adjacency mask. Conceptually, the mask changes which tokens may contribute information to a given token. A residual connection and a feed-forward network follow attention in each layer. This design combines content-dependent attention weights with a geometry-based restriction on attention eligibility. The notebook uses two such layers and four attention heads.

The locality is a modeling choice, not a claim that the implementation has the same computational cost as a sparse graph library. PyTorch's multi-head attention is called with a dense attention mask. The mask constrains communication, but the notebook does not document a sparse kernel or a measured speedup. This implementation detail is important when discussing efficiency.

2.5 Supervised Contrastive Learning

A classification loss encourages correct predictions. A contrastive objective shapes the embedding space by bringing same-class examples together and separating examples from different classes. Supervised Contrastive Learning uses class labels to identify positive examples within a batch [7]. SimCLR is a related self-supervised contrastive framework that learns representations from augmented views of the same image [19]; unlike SimCLR, the MANTA loss uses the available normal/anomaly labels to define positive pairs.

MANTA applies supervised contrastive loss to the output embedding of its cross-attention stage. It uses temperature 0.1 in the notebook implementation. Since the loss operates on examples within a minibatch, the number and composition of same-class examples in a batch affect the available positive pairs. The configured batch size is four. The notebook does not provide an ablation or a study varying batch composition, so the contribution of this term is not isolated in the reported results.

2.6 Feature Fusion and Cross-Attention

The MANTA network retains two global summaries: the mean of the projected patch tokens before graph refinement and the mean of graph-refined tokens. A learned sigmoid gate chooses a dimension-wise mixture of these summaries. This provides a direct path for the original projected features to contribute alongside the graph output.

Cross-attention then uses the fused vector as a query and the graph-refined patch tokens as keys and values. The resulting vector is passed to the binary classification head. The averaged attention weights from this operation are reshaped into the patch grid and used for the notebook's heatmap visualization. Attention weights are useful for inspecting which tokens received weight in this operation, but they are not automatically equivalent to causal explanations or verified defect boundaries.

2.7 Hybrid Training Objectives

The project combines four losses: cross-entropy classification, supervised contrastive learning, graph smoothness, and a reconstruction-style knowledge-fusion term. The classification term directly optimizes labels. The contrastive term shapes embeddings. The graph term penalizes squared differences between connected tokens. The knowledge term asks a decoder applied to the fused vector to reconstruct the detached pre-graph pooled vector.

The weights in the notebook are 1.0 for classification, 0.3 for contrastive, 0.2 for graph smoothness, and 0.2 for reconstruction. Combining these losses can express several preferences in one objective, but the magnitudes and gradients of the components may differ. No loss-component ablation or per-component validation results are included in the saved output. The weights should therefore be understood as the configuration used in this run, not as an empirically established optimum.

2.8 AUROC as an Image-Level Metric

The receiver operating characteristic curve plots the true-positive rate against the false-positive rate as a decision threshold varies. AUROC summarizes the area under this curve and can be interpreted as a ranking measure for positive and negative examples [8]. It is useful when a single operating threshold has not yet been selected, but it does not itself specify the operating point, class-specific precision, calibration, or the cost of false alarms.

For this report, image-level AUROC is calculated from the predicted anomaly probability and image label. Pixel-level AUROC would instead require per-pixel reference labels and a pixel score map. The notebook contains code for the latter when ground-truth mask folders are found, but its saved printed table contains only image-level AUROC and the train/test counts. The two metrics are not interchangeable.

2.9 Positioning of MANTA

MANTA brings together a pretrained ViT representation, graph-constrained patch attention, a gated combination of graph and non-graph summaries, cross-attention, and supervised objectives. Recent alternatives include the efficient student-teacher approach EfficientAD and the synthetic-feature approach SimpleNet [26, 29]. WinCLIP and AnomalyCLIP explore zero-shot anomaly detection using vision-language models and prompt learning [27, 28]. The supplied notebook comments contrast this classifier with a previous PatchCore baseline [1] and explain why labeled anomalous examples are now required. None of these methods is evaluated against MANTA in the supplied run; this report consequently avoids attributing the recorded AUROC to any individual component.

2.10 Literature-to-Implementation Summary

Table 2.1 connects broad ideas from the reviewed literature to the actual notebook components. It is a mapping of design rationale, not an experimental comparison.

Table 2.1: Relationship between background concepts and notebook components.

Concept	General role	MANTA implementation
Vision Transformer	Processes an image as patch tokens with attention	Frozen DINOv2-Large supplies patch features
Self-supervised pretraining	Reuses visual features without target labels for backbone training	Backbone remains frozen during category training
Spatial graph	Encodes selected relationships between patch locations	Symmetric spatial neighbor mask on the 16×16 grid
Supervised contrastive loss	Shapes representations using class identity	Applied to the cross-attention embedding
AUROC	Measures ranking of positive and negative examples	Computed from image-level anomaly probabilities

Chapter 3

Requirement Engineering

3.1 Stakeholders and Intended Use

The immediate stakeholder is the student developing and evaluating the course project. A faculty reviewer needs to understand the purpose, data flow, model configuration, and limits of the evidence. A potential end user, such as a researcher reviewing crop images, would need a clear score and a visual inspection aid, but the current notebook is not a validated field product. Its outputs should support experimentation rather than replace expert assessment.

The system is therefore framed as a research prototype. Any eventual operational use would require domain validation, a defined decision threshold, an assessment of false-positive and false-negative costs, and a workflow for human review. Those operational requirements are outside the recorded experiment and are identified here so that the report does not overstate deployment readiness.

3.2 Functional Requirements

The functional requirements below describe capabilities that are present in the notebook or required to interpret its outputs.

Table 3.1: Functional requirements and implementation status.

ID	Requirement	Implementation evidence
FR-01	Discover crop categories	When no explicit list is supplied, the notebook sorts and discovers category subfolders under the configured root. The saved run finds maize, paddy, soybean, and wheat.
FR-02	Locate normal and anomalous examples	A normal folder is sought using aliases `good`, `normal`, and `ok`; other image-bearing folders are grouped as defect types.
FR-03	Create train/test partitions	The notebook independently shuffles and splits each class using a 70:30 ratio with seed 42 and a minimum test count of one where possible.
FR-04	Prepare model input	Images are loaded, converted to tensors, resized to 224×224, and normalized with the ImageNet mean and standard deviation.

ID	Requirement	Implementation evidence
FR-05	Train a category model	A fresh MANTA graph model is trained per category using the labeled normal and aggregate anomaly training examples.
FR-06	Produce an image-level decision score	The classifier emits two logits; the softmax probability for the anomaly class is treated as the image score.
FR-07	Evaluate ranking performance	Image labels and anomaly probabilities are passed to AUROC computation when both classes appear in the evaluation subset.
FR-08	Create an attention visualization	Cross-attention weights are reshaped to the patch grid, resized for display, and overlaid on selected example images.
FR-09	Export results	Category scores and class counts are assembled into a tabular result and written as a CSV by the notebook.
FR-10	Support pixel-level evaluation when possible	The code searches for mask-folder aliases and contains pixel AUROC logic; the supplied saved result does not contain a verifiable pixel-AUROC value.

3.3 Non-Functional Requirements

Table 3.2: Non-functional requirements and current status.

ID	Quality attribute	Requirement and status
NFR-01	Reproducibility	Fix random seeds and record configuration. Seed 42 is used for Python, NumPy, and PyTorch; deterministic execution is not otherwise established.
NFR-02	Maintainability	Keep data-root and model parameters in one configuration object, rather than scattering values through the model code.
NFR-03	Resource awareness	Permit CUDA execution and offer smaller DINOv2 variants in the notebook notes for memory-constrained runs.
NFR-04	Interpretability support	Save or display attention overlays alongside example images; do not label the overlays as validated segmentations.
NFR-05	Robustness to incomplete data	Skip categories that do not contain both training labels and guard AUROC calculation when the evaluation set has only one label.
NFR-06	Portability	Allow category discovery and configurable data roots; the actual data path remains environment-specific and must be set by the user.

ID	Quality attribute	Requirement and status
NFR-07	Honest reporting	Preserve the distinction between executed values and metrics that are only implemented in code.

3.4 Input Data Requirements

The expected hierarchy places each crop category below a common data root. Within a category, the notebook looks for a split directory named `test`; if that directory is absent, it falls back to the category directory itself. Within the selected directory, one normal folder must be detectable using one of the normal aliases. Other child directories containing supported image extensions are interpreted as anomalous defect groups.

The loader accepts PNG, JPEG, BMP, and TIFF images. Class membership is inferred from folder location, not from a separate annotation file. All defect group images for a given category receive the anomaly label 1; normal images receive label 0. The individual defect names remain available for reporting and example titles. Pixel masks are an optional input path and are not required for image-level AUROC.

3.5 Operating and Environment Requirements

The notebook uses Python packages including PyTorch, TorchVision, NumPy, OpenCV, scikit-learn, Matplotlib, and tqdm. DINOv2 is loaded via PyTorch Hub. The saved execution output selects CUDA and records the DINOv2-Large checkpoint download. It also records that xFormers is unavailable; the run proceeds without it. The notebook does not record the precise GPU model, memory capacity, host operating system, or elapsed time, so those details cannot be included as measured environment properties.

The input size must remain divisible by the patch size 14 if the model is to form the intended patch grid. At 224 pixels, the grid size is $224/14 = 16$. Larger inputs change the token count and memory cost; smaller DINOv2 variants require matching backbone dimensions in the configuration. Any such change should be documented and treated as a new experimental configuration.

3.6 Use Cases

3.6.1 Train Category Model

The operator configures the dataset location and training parameters, runs category discovery, checks that both labels are available, and starts category training. The notebook prints periodic loss values and then evaluates the category-specific model against its held-out partition. The run requires access to the pretrained model weights and a compatible compute device.

3.6.2 Score an Image

During evaluation, an image passes through the frozen backbone and the trainable MANTA modules. The softmax probability of the anomaly class is returned as a continuous score. The saved notebook focuses on batch evaluation; it does not provide a separate packaged inference API, command-line interface, or threshold-based deployment decision.

3.6.3 Review Attention Overlay

The operator selects a stored test example and views the original image with a resized attention map. This may help explore which patch tokens were attended to, but the map is not a calibrated confidence explanation. If the task requires defect localization, the overlay must be compared with ground-truth masks using an appropriate pixel metric.

3.7 Constraints, Assumptions, and Risks

The main assumptions are that folder names correctly encode labels, images can be decoded, and the data partitions contain sufficient examples of both labels. The model's classifier is trained on labeled anomalous images; it may therefore fail on an unseen defect appearance. The split is random but performed within folders named `test`, and a single seed is used in the saved run.

Table 3.3 records risks that influence interpretation and follow-up.

Table 3.3: Project risks and practical mitigations.

Risk	Potential effect	Mitigation or follow-up
Folder labels are incorrect	Images may be assigned the wrong class	Inspect a sample from every folder and keep a data manifest
Only one split is evaluated	Performance can depend on the particular random partition	Repeat with multiple seeds and report variation
Train/test partition comes from a folder named test	It may not represent an independent evaluation cohort	Define and preserve a separate external test set
No confirmed pixel masks in output	Localization cannot be assessed quantitatively	Validate masks and report pixel-level results separately
Few samples per defect type	Per-subtype conclusions may be unstable	Report subgroup counts and collect more labeled examples
Large backbone memory demand	Training may fail on limited hardware	Use an appropriate GPU or a configured smaller backbone
Attention treated as explanation	Visual focus may be mistaken for true defect location	Compare attention with masks and expert judgments

3.8 Acceptance Criteria

For the scope of this report, the project is considered implemented when the notebook discovers the four expected categories, creates both-label training and test sets, completes the category training runs, emits image-level AUROC values, and saves example overlays. These criteria describe successful execution of the supplied run. They do not certify generalization, calibrated predictions, localization quality, or readiness for operational use.

Chapter 4

System Design

4.1 Architectural Overview

MANTA combines a frozen multi-depth visual encoder with trainable graph refinement, gated fusion, cross-attention, and binary classification. The diagram shows the raw-feature bypass and graph pathway; the only reported metric is image-level AUROC (I-AUROC). Figure 4.1 shows the high-level data flow.

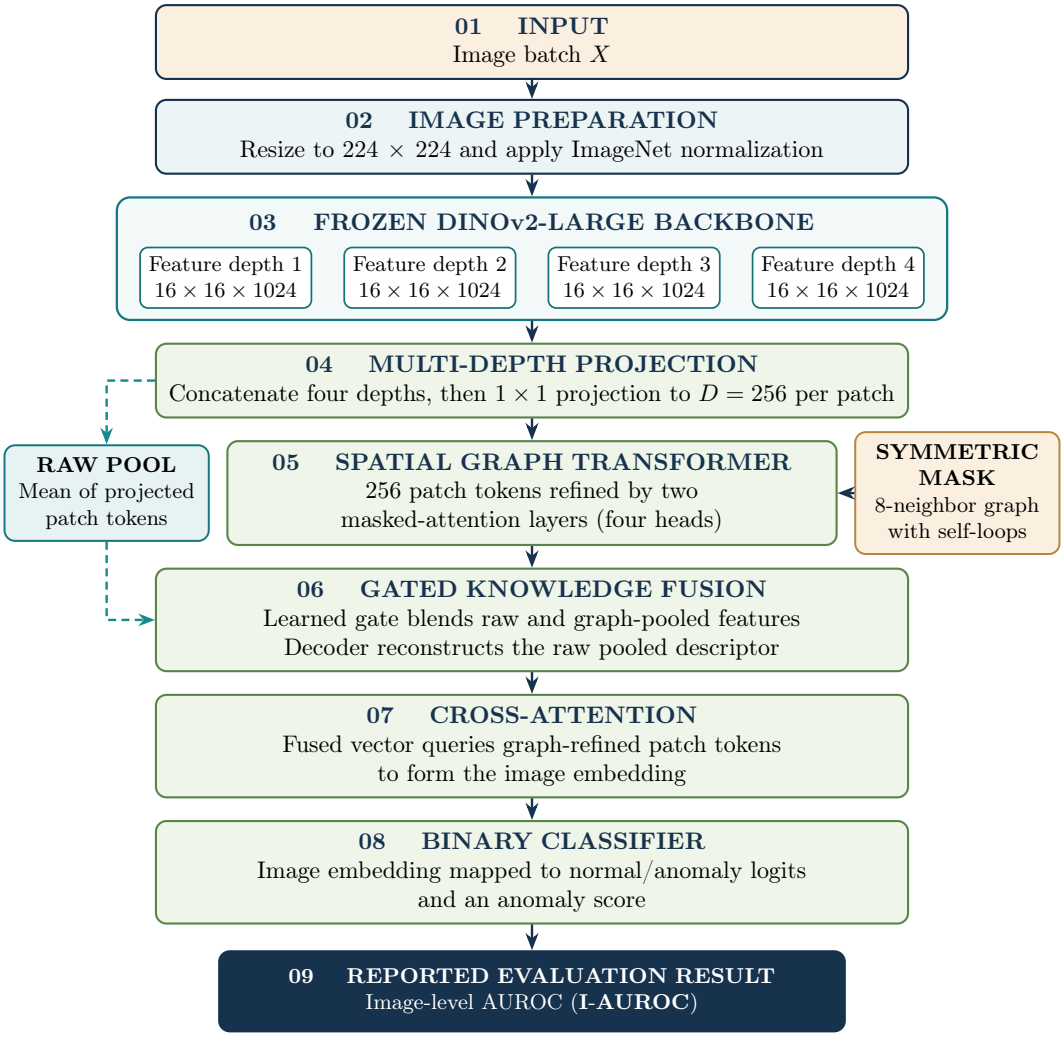


Figure 4.1: MANTA architecture with a raw-feature bypass into gated fusion, graph-masked refinement, and a single reported image-level I-AUROC result.

4.2 Subsystem Decomposition

4.2.1 Dataset Discovery

The discovery routine inspects each category directory and chooses the configured split directory when it exists. It identifies the normal directory from the aliases `good`, `normal`, or `ok`. All other child directories with supported images become named defect groups. This approach reduces manual category configuration but makes directory structure part of the data contract.

4.2.2 Image Dataset and Transform

Each dataset item contains an image tensor, a binary label, a textual defect type, the file path, and a mask tensor where mask loading is enabled. The image transform converts decoded image data to a tensor, resizes it to the configured square resolution, and normalizes each channel with ImageNet statistics. The training loader shuffles examples; the test loader preserves the requested order.

4.2.3 Frozen Feature Backbone

The DINOv2-Large model is loaded through PyTorch Hub, switched to evaluation mode, and frozen by setting its parameters not to require gradients. The feature extraction call returns the last four intermediate layers with spatial reshaping and normalization enabled. Gradient computation is disabled for this stage. In consequence, the notebook's optimizer updates the projection and downstream modules rather than the backbone.

4.2.4 Multi-Depth Projection

For each of the four selected depths, the backbone emits a feature map with 1024 channels in the configured variant. The maps are concatenated along the channel dimension, yielding 4096 channels at each patch location. A 1×1 convolution projects this concatenated feature to 256 dimensions. This operation learns a channel combination while retaining the 16×16 spatial layout.

4.2.5 Spatial Graph Transformer

The projected feature map is flattened to a sequence of 256 tokens. Each sequence position corresponds to a grid coordinate (r, c). The graph mask is computed once for the grid and reused for all images. For each node, the implementation selects its eight closest positions, includes itself in the selected set, symmetrizes the mask, and explicitly retains self-loops.

The saved notebook prints an adjacency tensor of shape 256×256 and an average row degree of 9.296875. The degree is larger than the nominal eight neighbors plus self because symmetrizing a directed nearest-neighbor selection can add incoming edges. A graph-transformer layer applies layer normalization, masked multi-head attention, a residual update, another normalization, and a feed-forward block with expansion ratio four, GELU, and dropout. Two such layers are stacked, with four attention heads and dropout 0.1 in the layer implementation.

4.2.6 Illustrative Patch Neighborhood

Figure 4.2 visualizes one query patch and its local neighborhood within the feature grid. The actual model contains 256 patch tokens, uses a symmetrized nearest-neighbor mask, and retains self-connections.

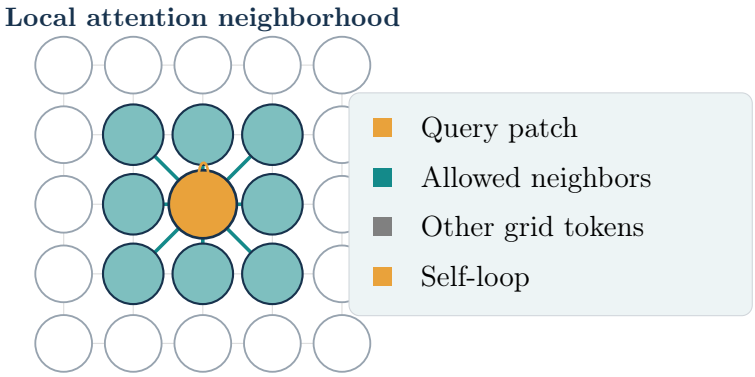


Figure 4.2: Illustrative 8-neighbor patch neighborhood. The center token attends to its selected nearby tokens and itself; the full 16×16 graph mask is symmetrized.

4.2.7 Knowledge Fusion

The model computes two global descriptors: the mean of the projected tokens before graph processing and the mean of graph-refined tokens. The fusion gate takes their concatenation, maps it to D dimensions, and applies a sigmoid. The resulting values determine the relative contribution of the two representations on a dimension-by-dimension basis. A learned linear decoder maps the fused representation toward the raw pooled feature for the reconstruction term.

4.2.8 Cross-Attention and Classifier

The fused vector becomes a single query token. The graph-refined patch sequence provides keys and values. Multi-head cross-attention returns a context vector and averaged weights over the patch positions. A residual connection from the query and layer normalization form the final image embedding. The classifier is a multilayer perceptron with a 128-unit hidden layer (half of D), GELU, dropout 0.2, and a two-logit output layer.

4.3 Feature and Graph Notation

Let $X \in \mathbb{R}^{B\times 3\times 224\times 224}$ be an input batch, where B is the batch size. Let $F^{(s)}$ denote the normalized feature map from depth s among the selected final transformer blocks. The projected patch map is

$$Z = W_{1\times 1}\,[F^{(1)}; F^{(2)}; F^{(3)}; F^{(4)}] + b, \tag{4.1}$$

where concatenation is along channels. Flattening spatial dimensions gives $Z_t \in \mathbb{R}^{B\times N\times D}$ with $N = 256$ and $D = 256$.

Let $A \in \{0,1\}^{N\times N}$ be the symmetric adjacency mask. For token pair (i, j), $A_{ij} = 1$ permits attention and $A_{ij} = 0$ masks the pair. Self-connections satisfy $A_{ii} = 1$. Since the implementation constructs the mask from fixed grid coordinates, A does not change from image to image.

4.4 Graph Transformer Layer

For a token sequence $H^{(\ell)}$, the first normalized representation is

$$U^{(\ell)} = \mathrm{LN}(H^{(\ell)}). \tag{4.2}$$

Masked multi-head self-attention is applied to $U^{(\ell)}$ as query, key, and value. The implementation supplies the inverse adjacency as the Boolean attention mask because masked positions are marked as unavailable. The residual attention update and feed-forward update can be written as

$$V^{(\ell)} = H^{(\ell)} + \mathrm{MHA}_A(U^{(\ell)}, U^{(\ell)}, U^{(\ell)}), \tag{4.3}$$

$$H^{(\ell+1)} = V^{(\ell)} + \mathrm{MLP}(\mathrm{LN}(V^{(\ell)})). \tag{4.4}$$

The same operation is repeated for two graph layers. The residual paths preserve a direct route for incoming feature information while allowing learned updates.

4.5 Gated Fusion and Cross-Attention Equations

The raw pooled descriptor and graph pooled descriptor are

$$z = \frac{1}{N}\sum_{i=1}^{N} Z_{t,i}, \qquad h = \frac{1}{N}\sum_{i=1}^{N} H_i. \tag{4.5}$$

The learned gate and fused vector are

$$g = \sigma(W_g[z; h] + b_g), \qquad f = g \odot z + (1 - g) \odot h. \tag{4.6}$$

Here σ is the sigmoid function and $\odot$ is elementwise multiplication. Cross-attention uses f as a single query and graph tokens as the key/value sequence. In standard scaled dot-product notation,

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \tag{4.7}$$

with multiple heads and learned projections. The notebook requests averaged attention weights, producing a length-N vector per image. This vector is reshaped to 16×16 before visualization.

4.6 Hybrid Loss

The complete objective recorded in the configuration is

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + 0.3\mathcal{L}_{\text{contrast}} + 0.2\mathcal{L}_{\text{graph}} + 0.2\mathcal{L}_{\text{knowledge}}. \tag{4.8}$$

4.6.1 Classification Loss

The classification term is two-class cross-entropy between the logits q_i and binary label y_i:

$$\mathcal{L}_{\text{cls}} = -\frac{1}{B}\sum_{i=1}^{B} \log \mathrm{softmax}(q_i)_{y_i}. \tag{4.9}$$

It supplies the direct label-learning signal for the normal/anomaly decision.

4.6.2 Supervised Contrastive Loss

Let e_i be the final embedding normalized to unit length and $s_{ij} = e_i^\top e_j$. For temperature $\tau = 0.1$, the implementation compares examples in the same minibatch, excludes each example from its own denominator, and treats other same-label examples as positives. In conventional notation,

$$\mathcal{L}_{\text{contrast}} = -\frac{1}{|I|}\sum_{i\in I}\frac{1}{|P(i)|}\sum_{p\in P(i)} \log \frac{\exp(s_{ip}/\tau)}{\sum_{a\neq i}\exp(s_{ia}/\tau)}, \tag{4.10}$$

where $P(i)$ is the set of same-label examples other than i, and I contains anchors with at least one positive. The notebook returns zero if a batch has no valid positive pairs. This makes batch composition relevant to the auxiliary objective.

4.6.3 Graph Smoothness Loss

For graph token $H_{b,i}$ and degree $d_i = \sum_j A_{ij}$, the smoothness term is

$$\mathcal{L}_{\text{graph}} = \frac{1}{BN} \sum_{b=1}^{B} \sum_{i=1}^{N} \frac{1}{d_i} \sum_{j=1}^{N} A_{ij} \, \|H_{b,i} - H_{b,j}\|_2^2 \,. \tag{4.11}$$

This corresponds to the notebook's per-node neighbor-average followed by a mean. It encourages neighboring patch representations to be similar. Since real defects can have sharp boundaries, the strength of this regularizer is a design choice and should be evaluated with ablations and localization data.

4.6.4 Knowledge Reconstruction Loss

Let r be the decoder applied to fused vector f and let $\text{sg}(z)$ denote the detached raw pooled descriptor. The reconstruction term is

$$\mathcal{L}_{\text{knowledge}} = \text{MSE}\big(r(f), \text{sg}(z)\big). \tag{4.12}$$

Detaching the target means gradients do not flow through the target representation in this term. The loss trains the fused representation and decoder to preserve information from the raw pooled vector.

4.7 Training and Evaluation Flows

The training flow runs independently for each category. A fresh MANTA model is constructed, trainable parameters are collected, and AdamW is initialized with learning rate 3×10^{-4} and weight decay 10^{-4}. Each batch passes through the forward model; the four loss components are combined; gradients are computed; and the optimizer updates trainable weights. The frozen backbone is reused and does not receive updates.

At evaluation, the model runs without gradient recording. For each test image, the anomaly softmax score is saved alongside its binary label. The patch attention weights are also reshaped and upsampled. Image AUROC is calculated over all test images in the category when both labels are present. Pixel AUROC is attempted only when collected pixel labels contain both classes, but no numerical pixel metric appears in the saved result table.

4.8 Algorithmic Outline

1. Discover categories and class folders; form a seeded split per normal and defect group.
2. For each category, construct a dataset, loaders, and a category-specific MANTA model.
3. For each epoch and batch, extract frozen DINOv2 features, project the four depths, and form patch tokens.
4. Refine tokens through two graph-transformer layers using the fixed symmetric adjacency mask.
5. Pool raw and graph tokens, fuse them with the learned gate, and apply cross-attention.
6. Compute logits and the four weighted loss terms; update trainable weights with AdamW.

7. Evaluate image anomaly probabilities, calculate I-AUROC, and retain example attention maps.
8. Write category-level results and generate visual examples.

4.9 Complexity and Resource Considerations

The input has 256 patch tokens. A dense attention operation over N tokens has an interaction matrix whose size grows with N^2; masked attention still operates through a dense PyTorch multi-head attention interface in the supplied implementation. Therefore, the spatial mask should be described as limiting allowed information flow, not as a demonstrated reduction in compute. Each graph layer also applies feed-forward projections over all tokens.

The pretrained DINOv2-Large backbone is the largest model component and its checkpoint download is recorded at approximately 1.13 GB in the notebook output. The saved run uses CUDA, but does not record GPU model, peak memory, training duration, or inference throughput. No speed or efficiency claim is made. Reducing image size, batch size, or backbone variant may help memory-constrained users, but each such change changes the experimental setup and should be logged.

Chapter 5

Implementation

5.1 Notebook Organization

The notebook is arranged as a sequence of executable stages. It installs or imports dependencies, sets a random seed, selects an available compute device, defines configuration values, discovers data, builds transforms and datasets, loads the pretrained backbone, constructs the graph mask and trainable modules, trains each category, evaluates metrics, writes a result table, and plots example attention overlays. This staged layout makes it possible to inspect intermediate configuration and category counts before training begins.

The notebook is the implementation artifact; this report does not replace or modify its code. The details below document its behavior as supplied. Some execution output reflects the saved run, while statements about possible alternate configurations describe notebook options rather than experiments that were performed.

5.2 Configuration Management

The configuration object includes the data root, category list, split-folder name, train ratio, minimum test count, input and patch sizes, backbone name and dimension, number of feature depths, graph dimensions, optimizer settings, loss weights, and output directory. Categories are automatically discovered in the saved run. The experiment uses a fixed random seed of 42 for Python, NumPy, and PyTorch.

The chosen backbone is `dinov2_vitl14`, with feature dimension 1024. Four final feature depths are selected and projected to dimension 256. The graph transformer has two layers and four attention heads; the spatial neighbor parameter is eight. Training runs for 15 epochs with batch size four, learning rate 3×10^{-4}, and weight decay 10^{-4}. The loss weights are 0.3, 0.2, and 0.2 for contrastive, graph, and knowledge reconstruction terms, respectively.

5.3 Category Discovery and Binary Labels

The loader supports a configured split directory named `test`, with a fallback to the category root if the split directory is missing. It locates the normal folder using aliases. For all other subdirectories it gathers supported image files recursively and preserves their names as defect types. Normal images are assigned label 0. Every defect group is assigned label 1 for the binary classifier.

The stratified splitter shuffles the paths independently for each class and assigns approximately 70 percent to training. It attempts to leave at least one sample for testing where possible. This class-wise splitting prevents an individual defect folder from being allocated entirely to one side when its size allows both partitions. It does not guarantee a fixed proportion for every tiny folder, and the saved output lists aggregate anomaly counts rather than counts for every defect subtype.

5.4 Image Transformation

The transform converts image pixels into a tensor, resizes to a square 224-pixel input, and applies channel-wise normalization with the ImageNet mean $(0.485, 0.456, 0.406)$ and standard deviation $(0.229, 0.224, 0.225)$. The multiple-of-14 image size is chosen to align with DINOv2's patch size. This creates 16 positions per side and 256 patch locations in total.

The resize operation standardizes the network input shape. The notebook does not report whether aspect ratios are preserved before resizing or whether crop-specific image augmentation is applied. No augmentation should therefore be assumed in interpreting the run. Future implementations may examine augmentation and image aspect handling, but would need to report those settings and rerun the evaluation.

5.5 Backbone Loading and Feature Use

The notebook loads the DINOv2 repository through PyTorch Hub and the `dinov2_vitl14` weights. The saved output records that the repository cache was used, the pretrained checkpoint was downloaded, and xFormers was unavailable. The model is set to evaluation mode and all parameters are frozen. The notebook then obtains four intermediate feature maps from the last transformer blocks and projects their concatenation.

The code contains a compatibility step that patches cached Python source files with a future-annotations import when needed. This is an implementation workaround tied to the execution environment, not part of the MANTA method itself. For a clean rerun, the model repository version, PyTorch/TorchVision versions, Python version, and checkpoint identity should be recorded; these are not all present in the notebook's result table.

5.6 Model Components

The trainable model includes a 1 by 1 convolution for multi-depth fusion, the graph transformer layers, the gated knowledge-fusion module, a cross-attention module, and a classification head. The graph mask is registered as a model buffer so it follows the model device and is included in model state. Each category receives a fresh model instance, which allows its trainable layers to specialize to that crop category while the frozen backbone is shared.

The graph transformer uses layer normalization before attention and before the feed-forward layer. Both stages use residual connections. The feed-forward subnetwork expands the feature width by four, applies GELU, a smooth activation function, and dropout, then projects back to the model width [23]. The classification head reduces the embedding from 256 to 128 dimensions, applies GELU and dropout, then produces two logits.

5.7 Optimization Loop

For each category with both normal and anomalous training images, the notebook creates loaders and initializes AdamW, which decouples weight decay from the gradient-based update, over parameters that require gradients [22]. Each epoch traverses the shuffled training loader. The model computes logits, embeddings, graph tokens, pooled features, reconstruction output, and attention weights. The hybrid loss is evaluated, gradients are reset, backpropagation is performed, and the optimizer steps.

Periodic logs are emitted at approximately one-fifth of the total number of epochs. For a 15-epoch run, the saved logs show checkpoints at epochs 3, 6, 9, 12, and 15. The scalar displayed is the mean of the total batch loss values for that epoch. The output does not provide a full per-batch trace or separate logged curves for each loss component.

5.8 Evaluation and Export

Evaluation runs with the model in evaluation mode and gradients disabled. The positive-class softmax probability is the image score. The category I-AUROC is calculated using the image labels and those probabilities. If the evaluation labels contain only one class, the code returns an undefined value instead of attempting to calculate AUROC.

The notebook also obtains averaged cross-attention weights and reshapes them to a 16 by 16 map. It resizes this map for display and overlays it on selected images. The result table is exported as a CSV containing category, image AUROC, pixel AUROC when available, and class counts. In the saved notebook output, the printed result table shows I-AUROC and the counts but does not show P-AUROC. This discrepancy is handled conservatively in the report: no pixel metric is claimed.

5.9 Outputs Produced

The notebook's intended output artifacts are the category-wise result CSV and example overlays named by category. Training progress is printed to the notebook output. The saved execution includes four category figures, each with multiple image/map panels. The report does not embed those sample images because the supplied notebook provides the visualizations as generated notebook outputs rather than as a separately named asset collection for this TeX project.

5.10 Error Handling and Operational Notes

The notebook includes checks and guidance for several common conditions. A missing data root raises an assertion with a path reminder. A category without either training label is skipped. The evaluation code handles missing pixel masks by leaving the pixel metric unavailable. The notebook notes that anomalous mask warnings should be checked, while a missing mask for a normal example is expected. If memory is insufficient, a smaller backbone, reduced image size that remains divisible by 14, or a smaller batch size is suggested.

These safeguards make a notebook run more diagnosable, but they do not substitute for verifying all input paths and labels. Before rerunning on a new machine, the user should confirm the exact data hierarchy, image decoding, category names, model download access, available device, and

output directory. Configuration changes should be recorded so that scores from different runs are not conflated.

5.11 Reproducibility Record

A minimally useful record of a MANTA run includes: source notebook identity; dataset version and folder manifest; split seed and counts; image size and normalization; backbone variant and checkpoint; graph dimensions and neighbor value; optimizer and loss coefficients; software versions; compute device; per-category training loss logs; and saved metric outputs. The notebook already supplies several of these fields, including seed, model settings, device selection, category counts, and I-AUROC. It does not include all environment and dataset provenance details, so an independent reproduction should add them explicitly.

Chapter 6

Results and Discussion

6.1 Evaluation Protocol in the Saved Run

The notebook discovers four crop categories and performs a seeded, class-wise split with a 70:30 training-to-test ratio. The path configuration points to the category subfolder named `test`; the notebook then creates training and test subsets from the images it finds there. The saved run uses 15 epochs and evaluates each category model once on its held-out subset. The output does not document an additional validation partition or a second independent test collection.

For every test image the classifier returns two logits. The softmax probability corresponding to the anomaly class is used as a continuous ranking score. The saved run prints image-level AUROC and training/test counts. It does not select a decision threshold or provide accuracy, precision, recall, F1, calibration, confidence intervals, or a multiple-seed summary.

6.2 Category-Level Scores

Table 6.1 reproduces the saved category table. The test counts reflect the generated partitions, not a separate dataset supplied for the report.

Table 6.1: Saved category-wise image-level results.

Category	Train normal	Train anomaly	Test normal	Test anomaly	I-AUROC (%)
Maize	140	70	60	30	99.00
Paddy	140	69	60	31	99.25
Soybean	140	70	60	30	99.39
Wheat	140	70	60	30	98.22
Total	560	279	240	121	—
Reported mean	—	—	—	—	98.96

The aggregate counts correspond to 839 training images and 361 test images across the four category runs, for 1,200 image instances in the printed split counts. This total is arithmetic over the reported counts, not a claim about the complete source archive beyond the images included in this run.

6.3 Per-Category Discussion

6.3.1 Maize

The saved split contains 140 normal and 70 anomalous training examples, followed by 60 normal and 30 anomalous test examples. The notebook reports an I-AUROC of 99.00%. Defect groups discovered for maize are crack, heat damage, insect damage, mildew, mold, multi-anomalies, spot, and sprouting. The saved output does not provide a separate score for any of these defect groups, so the category result should be interpreted as an aggregate ranking result across all anomaly examples.

6.3.2 Paddy

Paddy has 140 normal and 69 anomalous training examples, with 60 normal and 31 anomalous test examples. Its recorded I-AUROC is 99.25%. The discovered groups are crack, insect damage, mold, multi-anomalies, shriveling, spot, and sprouting. The anomaly count differs by one from the other categories in the saved split. The notebook's aggregated result does not show whether performance is consistent across these seven defect groups.

6.3.3 Soybean

The soybean run uses 140 normal and 70 anomalous training examples and 60 normal and 30 anomalous test examples. Its recorded I-AUROC is 99.39%, the highest of the four saved category values. The discovered defect groups are crack, insect damage, mold, multi-anomalies, spot, sprouting, and swelling. The high aggregate score is an observation on one held-out split and does not establish robustness across new sources, environments, or seeds.

6.3.4 Wheat

The wheat split has the same aggregate counts as maize and soybean: 140 normal and 70 anomalous training examples, with 60 normal and 30 anomalous test examples. The saved I-AUROC is 98.22%, the lowest among the four categories but still high on this partition. The discovered groups are black embryo, crack, insect damage, mold, spot, and sprouting. The saved output does not provide per-defect rankings or a wheat-specific threshold.

6.4 Training Loss Trace

The notebook prints mean total training loss at five checkpoints for each category. Table 6.2 transcribes those values. These are training losses, not validation losses, and are not direct estimates of generalization.

Table 6.2: Mean total training loss at the logged epochs.

Category	Epoch 3	Epoch 6	Epoch 9	Epoch 12	Epoch 15
Maize	1.5111	0.8205	0.6580	0.3605	0.3085
Paddy	1.7996	0.9267	0.7330	0.5416	0.5066
Soybean	1.8178	1.0175	0.6013	0.5796	0.5525
Wheat	1.8510	1.1312	0.7285	0.5570	0.3733

All four traces decrease at the displayed checkpoints. This confirms that the reported optimization loss became smaller over the run, but it does not establish that the model avoided overfitting. A validation curve, repeated training, and comparison between training and held-out behavior would be needed to make that assessment.

6.5 Interpretation of AUROC

An I-AUROC close to 100% means that, for this test partition, the score generally ranks anomalous images above normal images. AUROC is threshold-independent in the sense that it summarizes ranking across thresholds. It does not indicate which threshold should be used in practice. For example, the chosen operating threshold would depend on the relative consequences of missing an anomaly and flagging a normal image, neither of which is specified in the saved output.

The four category values range from 98.22% to 99.39%, and the notebook reports a mean of 98.96%. Since category test sizes are similar, the unweighted category mean is easy to interpret as a summary across the four outputs. It should not be treated as a confidence interval or as a metric on a pooled, independent collection. There is no repeated-seed variation reported.

6.6 Confusion Matrix, Accuracy, and F1-Score

The notebook's saved output contains category-wise I-AUROC values and class counts, but not per-image hard predictions. Therefore it cannot provide measured threshold-dependent metrics. For demonstration, the report uses the illustrative pooled operating point specified in the supplied MANTA reference: 240 normal and 121 anomalous images, with TN=236, FP=4, FN=3, and TP=118. This example gives 354/361 correct predictions (98.06% accuracy). The values are illustrative, not outputs recovered from the notebook, and are not crop-specific results.

The aggregate notebook I-AUROC mean of 98.96% is a separate ranking metric and does not determine this illustrative confusion matrix. Actual thresholded results require per-image scores and labels, grouped by crop, plus a stated threshold.

For a binary task, the confusion matrix compares the ground-truth label with the predicted label. Here, anomaly is the positive class. A true negative (TN) is a normal image predicted normal; a false positive (FP) is a normal image predicted anomalous; a false negative (FN) is an anomalous image predicted normal; and a true positive (TP) is an anomalous image predicted anomalous.

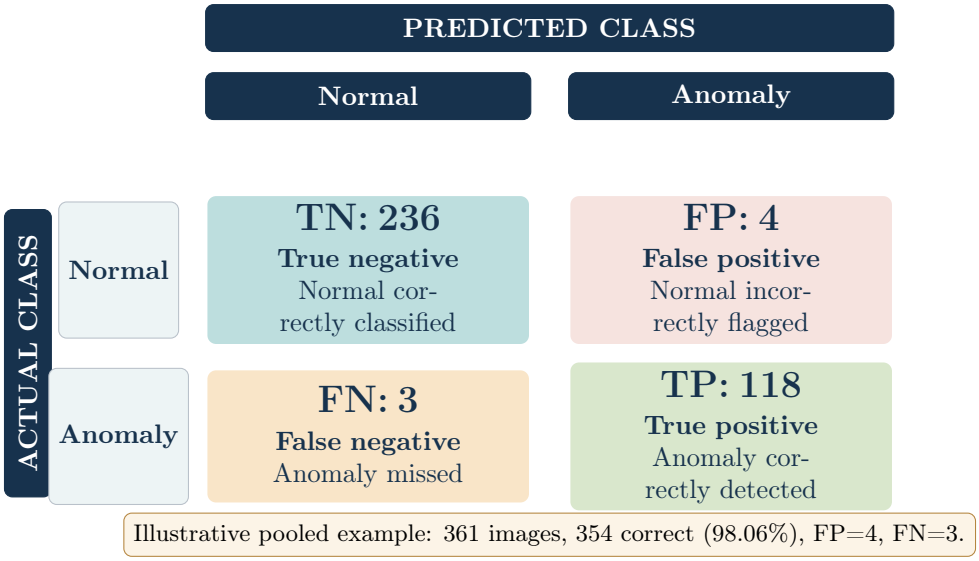


Figure 6.1: Illustrative pooled confusion-matrix example from the supplied reference. It is not a measured result from the saved notebook.

At a selected threshold t, the model converts anomaly probability s_i into a class prediction $\hat{y}_i = 1$ when $s_i \geq t$, and $\hat{y}_i = 0$ otherwise. The requested metrics are then calculated from the four confusion-matrix counts:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \tag{6.1}$$

$$\text{Precision}_{\text{anom}} = \frac{TP}{TP + FP}, \tag{6.2}$$

$$\text{Recall}_{\text{anom}} = \frac{TP}{TP + FN}, \tag{6.3}$$

$$\text{Specificity} = \frac{TN}{TN + FP}, \tag{6.4}$$

$$F1_{\text{anom}} = 2 \cdot \frac{\text{Precision}_{\text{anom}}\,\text{Recall}_{\text{anom}}}{\text{Precision}_{\text{anom}} + \text{Recall}_{\text{anom}}} = \frac{2TP}{2TP + FP + FN}. \tag{6.5}$$

The F1-score in this report is defined for the anomalous class as positive. If a prediction set contains no positive predictions, precision has a zero denominator; the calculation must define a consistent convention, such as scikit-learn's `zero_division=0`. Accuracy alone can hide missed anomalies when normal examples are more numerous, so precision and recall should be interpreted alongside it.

6.6.1 Threshold and Metric Reporting

The saved notebook computes I-AUROC from continuous probabilities, which does not require a single classification threshold. Confusion-matrix metrics do require one. A threshold should ideally be selected using a validation set and frozen before the independent test set is evaluated. If no validation set exists, a threshold of 0.5 may be reported as a clearly stated reference point, but it should not be described as optimized for this dataset.

Table 6.3: Metrics calculated from the illustrative pooled confusion matrix in Table 6.4. Values other than the confusion counts are derived from that example, not measured notebook predictions.

Metric	Definition	Value
Accuracy	$(TP + TN)/(TP + TN + FP + FN)$	98.06%
Precision	$TP/(TP + FP)$	0.9672
Recall / Sensitivity	$TP/(TP + FN)$	0.9752
Specificity	$TN/(TN + FP)$	0.9833
F1-score	$2TP/(2TP + FP + FN)$	0.9712
False-positive rate	$FP/(FP + TN)$	0.0167
False-negative rate	$FN/(FN + TP)$	0.0248

Table 6.4: Illustrative pooled confusion counts used to calculate the metrics in Table 6.3.

Illustrative scenario	TN	FP	FN	TP
Pooled illustrative example	236	4	3	118

The illustrative matrix contains 240 actual normal images and 121 actual anomalous images. It yields 239 predicted normal images and 122 predicted anomalies, for 354 correct predictions out of 361. This scenario is copied from the supplied reference and is not a substitute for thresholded outputs from the current notebook.

6.6.2 Recalculation After a Notebook Rerun

The following calculation can be applied inside the evaluation function after `img_scores` and `img_labels` have been collected. It uses threshold 0.5 only as a reference setting. The resulting values should be copied into Tables 6.3 and 6.4 only after a fresh run on the intended data split.

```
from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score,
    recall_score, f1_score
)

threshold = 0.5
predicted = (np.asarray(img_scores) >= threshold).astype(int)
actual = np.asarray(img_labels, dtype=int)
tn, fp, fn, tp = confusion_matrix(
    actual, predicted, labels=[0, 1]
).ravel()
accuracy = accuracy_score(actual, predicted)
precision = precision_score(actual, predicted, zero_division=0)
recall = recall_score(actual, predicted, zero_division=0)
f1 = f1_score(actual, predicted, zero_division=0)
```

The values above are not available for the current saved execution because its output retained AUROC and sample counts but did not print per-image predictions or threshold-based counts. Re-running the notebook is necessary to obtain genuine values.

6.7 Attention Maps and Pixel-Level Evidence

The visualization routine displays test examples with their cross-attention maps resized and blended over the images. These visual outputs support inspection of attended patch locations, but the saved results do not establish that highlighted regions match defects. The notebook contains optional mask lookup and pixel-level AUROC code, but the saved output table has no verifiable pixel-level score. Localization claims require verified mask alignment and quantitative comparison against pixel annotations.

6.7.1 Saved Cross-Attention Figures

The four figures are ordered maize, paddy, soybean, then wheat (Figures 6.2–6.5). The notebook contains one visualization for each crop category. The test loader places normal images before anomalous images, and the visualization function selects the first examples; therefore, the displayed samples are labeled `good`. These images show attention on normal samples and are qualitative illustrations, not evidence of defect localization.

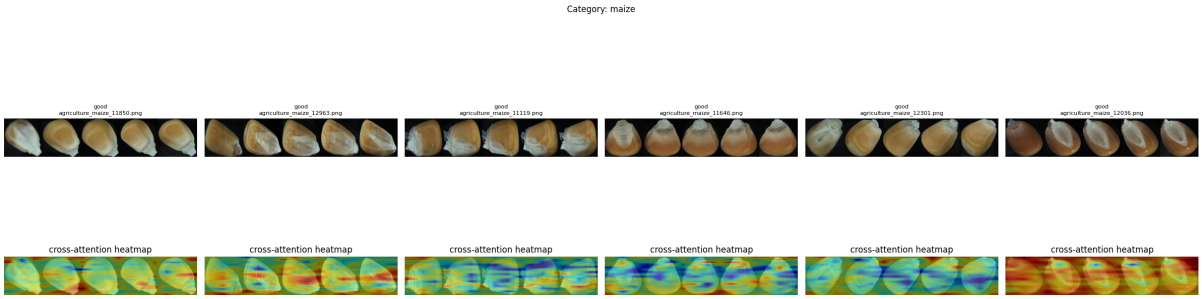


Figure 6.2: Maize: six normal-class input composites and their cross-attention overlays.

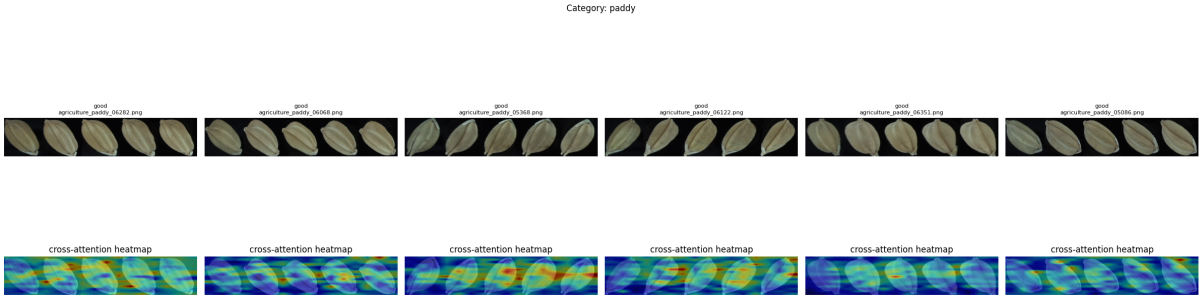


Figure 6.3: Paddy: six normal-class input composites and their cross-attention overlays.

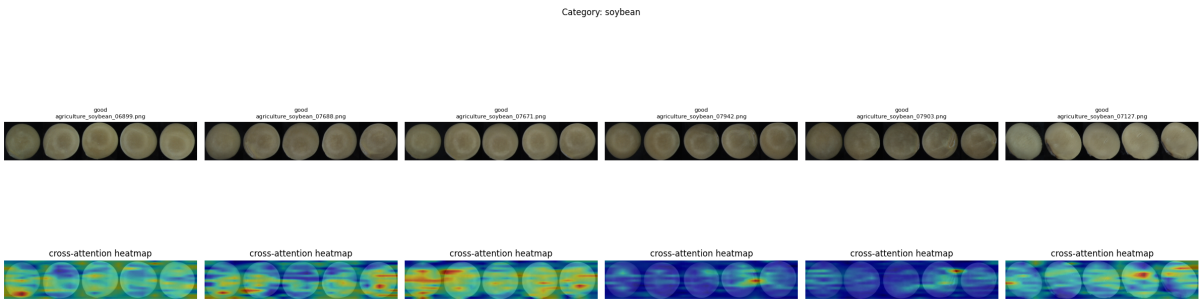


Figure 6.4: Soybean: six normal-class input composites and their cross-attention overlays.

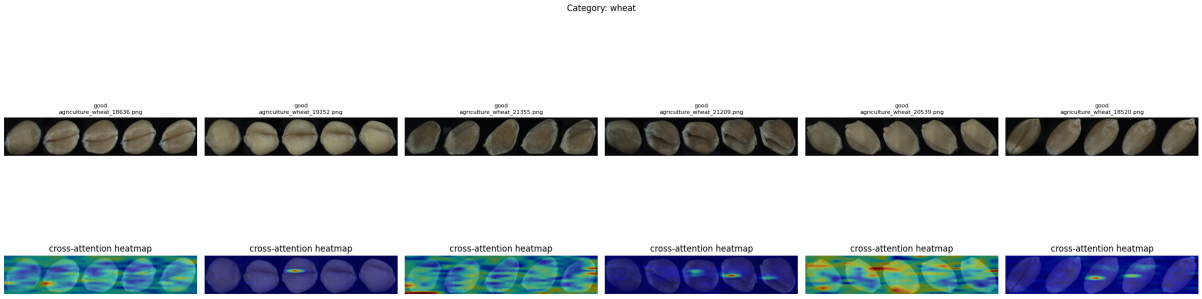


Figure 6.5: Wheat: six normal-class input composites and their cross-attention overlays.

6.8 Published MANTA Benchmark Results

Published-paper results below are separated from the four-category notebook experiment because their dataset and protocols differ.[9]

6.8.1 Dataset Profile

Table 6.5: Published MANTA benchmark profile.[9]

Aspect	Published dataset detail
Coverage	137,338 images, 38 categories, five domains
Views	Five views per object
Anomaly localization	8,617 anomalous images with pixel-level annotations
Text resources	875 declarative-knowledge words and 2,000 multiple-choice questions

6.8.2 Single-View Results

Table 6.6: Single-view results across five domains (I-AUROC/P-AUROC, %).[9]

Method	View evaluation	Object evaluation
RD	82.0 / 90.0	85.6 / 90.0
PatchCore	93.7 / 95.7	95.2 / 95.8
CDO	91.2 / 94.7	91.3 / 94.4
DMAD	82.6 / 89.6	86.1 / 89.7
SimpleNet	87.1 / 86.4	90.9 / 87.5

6.8.3 Multi-View Results

Table 6.7: Published multi-view MANTA results by domain (I-AUROC/P-AUROC, %).[9]

Method	Agriculture	Medicine	Electronics	Mechanics	Groceries	Average
RD	86.9 / 89.5	94.3 / 96.1	94.9 / 98.5	96.1 / 98.7	81.1 / 87.3	90.6 / 94.0
PatchCore	94.1 / 93.0	96.8 / 96.2	96.8 / 98.7	96.6 / 98.6	91.0 / 91.8	95.0 / 95.7
CDO	90.7 / 90.4	95.4 / 95.5	92.3 / 98.6	91.3 / 98.4	87.8 / 89.5	91.5 / 94.5
DMAD	88.5 / 89.7	95.1 / 95.8	94.2 / 98.3	93.3 / 98.3	80.0 / 86.3	90.2 / 93.7

Table 6.8: Published MANTA benchmark comparisons and interpretation.[9]

Comparison	Summary
Single-view	PatchCore has the strongest average in both view-eval and object-eval; grouping views generally raises I-AUROC more than P-AUROC.
Multi-view	PatchCore has the highest reported average I-AUROC/P-AUROC (95.0/95.7); agriculture and groceries are comparatively challenging domains.
Multi-class	A shared model has lower reported average AUROCs than the best category-specific results, indicating a performance trade-off when one model covers multiple object classes.

6.8.4 Multi-Class Results

Table 6.9: Published multi-class MANTA averages (%).[9]

Method	Average I-AUROC	Average P-AUROC
UniAD	82.4	89.4
CRAD	88.6	91.6
HGAD	85.5	88.6

6.8.5 Text-Prompt Results

Table 6.10: Published one-shot text-prompt evaluation summary on a balanced 1,000-image test set.[9]

Method	Evaluation setting	Reported summary
WinCLIP	One-shot	Average I-AUROC $> 75\%$ and average P-AUROC $> 90\%$
PromptAD	One-shot	Evaluated; no average score transcribed in the supplied reference
VCP-CLIP	One-shot	Evaluated; no average score transcribed in the supplied reference

6.8.6 Visual-Language Results

Table 6.11: Published average visual-language accuracy (mean ± reported standard deviation, %). The ± values denote variability, not additive gains.[9]

Model	Option accuracy	Question accuracy
LLaVA 1.5	47.8 ±9.0	1.7 ±2.0
LLaVA-NeXT	47.4 ±10.0	1.4 ±2.0
InternVL2	51.6 ±1.7	0.7 ±0.4
Molmo	51.5 ±2.6	0.8 ±3.7
MANTA paper baseline	52.7 ±13.3	0.5 ±3.7

6.8.7 Qualitative Findings

Table 6.12: Published qualitative observations and practical implications.[9]

Observation	Practical implication
Small defects such as insect holes and mold spots can be localized, while large defects may be only partly covered.	Evaluate small localized defects separately from large structural defects.
Predictions may be more consistent across views, but resizing multi-view composites can reduce spatial detail and introduce localization errors.	Report the resize policy and inspect mask quality alongside image-level AUROC.
Attention maps show relative responses, not calibrated anomaly probability or causal evidence.	Validate localization against pixel-level masks before making segmentation claims.

6.9 Threats to Validity

6.9.1 Data Split

The data loader splits images found under the category folders named `test`. This creates a hold-out partition within those folders, but it is not the conventional protocol of training on a designated training set and evaluating on a separately preserved test set. The reported results should be read in light of that implementation.

6.9.2 Sample and Category Coverage

The reported counts are modest and are aggregated across multiple defect types. No per-defect AUROC, per-source breakdown, or confidence interval is shown. A high overall ranking score can conceal weaker performance on an individual defect group or image acquisition condition.

6.9.3 Model Selection and Reproducibility

Only one random seed and one set of loss weights are represented in the saved run. No ablation study compares the full network with versions lacking graph attention, contrastive loss, or the fusion module. As a result, the notebook demonstrates a functioning combined architecture but does not establish which component accounts for any performance difference.

6.9.4 Metric Coverage

Only image-level AUROC is present in the printed result table. A deployment threshold, calibration measure, confusion matrix, class-specific precision/recall, inference time, and pixel-level localization score are not reported. These omissions limit operational and localization conclusions.

6.10 Summary of Findings

The published MANTA benchmark results summarized in this chapter provide external context only; their dataset, tasks, and evaluation protocols differ from the four-category notebook ex-

periment, so the scores are not directly comparable. The notebook execution completes the four category runs, prints declining training loss checkpoints, and reports image-level AUROC values in the 98.22–99.39% range. The reported mean is 98.96%. These are promising initial results for the generated partitions. The main qualification is methodological: the test subset is made from the folders named `test`, only one seeded split is shown, and no verifiable pixel AUROC accompanies the attention visualizations. Future evaluation should address these issues before interpreting the scores as evidence of broader generalization.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This project documents MANTA, a supervised agricultural image-anomaly pipeline that combines frozen DINOv2-Large features, multi-depth feature projection, spatially masked graph attention, gated fusion, cross-attention, and binary classification. The notebook trains one category-specific model for maize, paddy, soybean, and wheat. Its hybrid objective adds supervised contrastive, graph-smoothness, and reconstruction terms to cross-entropy classification.

The saved execution reports I-AUROC values of 99.00% for maize, 99.25% for paddy, 99.39% for soybean, and 98.22% for wheat, with a notebook-reported mean of 98.96%. The recorded loss values decrease at the printed checkpoints. These results show that the configured pipeline ran and separated labels well on the generated hold-out partitions. They do not by themselves establish performance on independent data or prove the contribution of each architectural component.

The data-splitting detail is especially important: the notebook uses images under a folder named `test` as its source and makes a new stratified 70:30 split from those images. The report therefore describes the result as an evaluation on the notebook's held-out partition, not as a result on a separate external test set. Likewise, cross-attention overlays are presented as qualitative visualizations. The saved output does not contain a verifiable pixel-level AUROC, so no quantitative localization claim is made.

7.2 Recommended Next Steps

The next steps are ordered to improve the reliability of the evidence before expanding the architecture.

1. **Define the evaluation protocol.** Preserve an independent test set that is not used for training, model selection, or parameter adjustment. Keep the current split only as a development experiment.
2. **Repeat the experiment.** Run multiple documented seeds and report category-wise mean and variability. Preserve split manifests so the runs can be reconstructed.
3. **Audit the data.** Inspect file counts, image decoding, class-folder labels, duplicates, and masks. Add per-defect counts to the exported report.
4. **Establish a simple baseline.** Compare the model with a frozen-feature linear classifier

and an appropriate anomaly-detection baseline using the same independent split. Report protocol and scores for each.

5. **Run ablations.** Remove or vary graph attention, multi-depth fusion, contrastive loss, graph smoothness, and knowledge reconstruction one at a time. This tests whether each design component is useful.
6. **Validate localization.** Confirm the mask hierarchy and image-mask pairing, save P-AUROC if calculated, and consider additional region metrics appropriate to the intended use.
7. **Assess operating behavior.** Select thresholds on validation data and report confusion matrices, precision, recall, and the consequences of false positives and false negatives.
8. **Measure deployment costs.** Record hardware, wall-clock training time, inference throughput, and peak memory before claiming that the system is practical for a particular workflow.

7.3 Final Remarks

The present implementation is a useful research prototype and a concrete starting point for combining pretrained visual features with spatial graph reasoning. The report preserves the positive recorded results while making the evaluation boundaries explicit. The strongest next contribution would be a more rigorous data protocol and component-wise experiment, not an unsupported performance claim. A verified external test set, reproducible splits, and pixel-level ground truth would make subsequent conclusions more informative for both researchers and potential users.

Appendix A

Run Configuration and Dataset Inventory

This appendix consolidates the settings and category information printed by the supplied notebook. Values in the tables describe the recorded configuration; suggestions for alternative settings are not results from this run.

A.1 Model and Training Configuration

Table A.1: Configuration values used in the saved MANTA run.

Parameter	Value	Description
Split folder	`test`	Category subfolder selected as the source of images when present
Train ratio	0.70	Fraction assigned to training independently per class
Minimum test per class	1	Minimum held-out item attempted for each class when possible
Random seed	42	Applied to Python, NumPy, and PyTorch random generators
Input size	224×224	Square network input resolution
Patch size	14	Produces 16 positions on each spatial axis
Backbone	`dinov2_vitl14`	Frozen DINOv2-Large transformer
Backbone width	1024	Per-depth feature channel dimension
Feature depths	4	Number of final transformer depths extracted
Projected width	256	Dimension after 1×1 feature projection
Graph layers	2	Number of graph-transformer layers
Attention heads	4	Number of heads per attention module
Spatial k	8	Number of nearest grid neighbors requested before symmetrization

Parameter	Value	Description
Batch size	4	Number of training images per minibatch
Epochs	15	Training epochs for each category
Optimizer	AdamW	Optimizer used for trainable parameters
Learning rate	3×10^{-4}	Initial configured learning rate
Weight decay	10^{-4}	AdamW weight-decay coefficient
Contrastive weight	0.3	Coefficient on supervised contrastive loss
Graph weight	0.2	Coefficient on graph-smoothness loss
Knowledge weight	0.2	Coefficient on reconstruction loss
Compute device	CUDA	Device printed by the saved run

A.2 Categories and Defect Groups

The notebook identifies a normal class in each crop category and treats each listed defect folder as a positive example in the binary task. The names below are transcribed from the discovery output.

Table A.2: Defect-group directory names found in the saved run.

Crop category	Discovered defect groups
Maize	crack; heat damage; insect damage; mildew; mold; multi_anomalies; spot; sprouting
Paddy	crack; insect damage; mold; multi_anomalies; shriveling; spot; sprouting
Soybean	crack; insect damage; mold; multi_anomalies; spot; sprouting; swelling
Wheat	black embryo; crack; insect damage; mold; spot; sprouting

A.3 Partition Counts

Table A.3: Aggregate label counts from category discovery.

Category	Train normal	Train anomaly	Test normal	Test anomaly
Maize	140	70	60	30
Paddy	140	69	60	31
Soybean	140	70	60	30
Wheat	140	70	60	30
Total	560	279	240	121

These numbers describe only the images included in the saved split. They do not specify the number of images in every individual defect folder, because the notebook output shown for the report aggregates all defect types into the anomaly count.

A.4 Training Checkpoint Values

Table A.4: Training loss checkpoints reproduced from the notebook log.

Category	3	6	9	12	15
Maize	1.5111	0.8205	0.6580	0.3605	0.3085
Paddy	1.7996	0.9267	0.7330	0.5416	0.5066
Soybean	1.8178	1.0175	0.6013	0.5796	0.5525
Wheat	1.8510	1.1312	0.7285	0.5570	0.3733

The column headings are epoch numbers. The logged value is mean total training loss at that checkpoint; no validation-loss values are shown in the saved output.

A.5 Expected Folder Layout

The loader expects category folders and class subfolders. In schematic form, the structure is:

```
<data-root>/
  maize/
    test/
      good/
      crack/
      heat damage/
      ...
  paddy/
    test/
      good/
      crack/
      ...
  soybean/
    test/
      good/
      swelling/
      ...
  wheat/
    test/
      good/
      black embryo/
      ...
```

This diagram expresses the loader's expected organization, not a guarantee that every uploaded directory exactly matches the example. The notebook includes fallback behavior when a category's `test` directory is absent, and mask directories are searched separately using aliases such as `ground_truth`, `gt`, `masks`, or `mask`.

Appendix B

Algorithms, Verification, and Reporting Checklist

B.1 Category Training Pseudocode

The following pseudocode summarizes the behavior of the supplied training routine. It is intended to clarify the sequence of operations and does not introduce additional experiments.

1. Set the random seeds and select CUDA when available.
2. Read the configuration and discover category directories.
3. For each category, locate the normal directory and all defect directories.
4. Shuffle and split each class into training and held-out subsets.
5. If either training label is missing, skip that category.
6. Create training and evaluation data loaders.
7. Initialize the frozen DINOv2 backbone and a new trainable MANTA module set.
8. Build the fixed adjacency mask for the patch grid.
9. For each epoch and each training batch:
   (a) Extract four frozen backbone feature maps.
   (b) Concatenate and project multi-depth features into patch tokens.
   (c) Apply graph-transformer layers under the adjacency mask.
   (d) Pool features, compute gated fusion, and attend from the fused query to graph tokens.
   (e) Compute classification, contrastive, graph, and reconstruction terms.
   (f) Backpropagate the weighted loss and update trainable weights with AdamW.
10. Evaluate anomaly probabilities and store image labels.
11. Compute image AUROC when the held-out set contains both labels.
12. Generate attention maps and save category result/visualization outputs.

B.2 Suggested Verification Cases

The notebook is intended to be run in an environment where the dataset and pretrained weights are accessible. The following verification cases provide a practical checklist for a future rerun. The status column distinguishes behavior visible in the supplied implementation from checks that still require explicit verification.

Table B.1: Verification cases for a reproducible rerun.

ID	Case	Expected behavior	Evidence/status
TC-01	Discover category folders	Four expected crop category names are listed	Observed in saved discovery output
TC-02	Detect normal folder	Normal alias is found and assigned label 0	Implemented; verify each new data root
TC-03	Detect defect groups	Non-normal image folders are collected as positive examples	Observed for four categories
TC-04	Split labels	Each class is represented in both subsets when its sample count permits	Counts printed for saved split
TC-05	Validate image transform	Tensor has three channels and configured spatial dimensions	Transform configured; inspect with a new dataset
TC-06	Check patch grid	224 input and patch size 14 produce 256 tokens	Observed in saved execution output
TC-07	Check graph mask	Mask is square, includes self-loops, and uses grid neighbors	Shape and average degree printed
TC-08	Train category model	Loss log appears through epoch 15 for each supported category	Observed for all four categories
TC-09	Handle missing training class	Category is skipped with an explanatory message	Implemented branch; no such saved category
TC-10	Evaluate I-AUROC	Numeric result appears when both labels exist	Observed for all four saved categories
TC-11	Evaluate pixel AUROC	Numeric result appears only with valid pixel labels from masks	Code exists; saved metric not verifiable
TC-12	Export results	CSV contains category scores and counts	Export path printed by notebook
TC-13	Inspect attention visualization	Original image and attention overlay are rendered per category	Four figure outputs recorded
TC-14	Reproduce exact run	Same split and values can be recreated from exact assets/environment	Requires dataset/version and environment manifest

B.3 Rerun Checklist

Before training:

- Confirm the dataset root and verify category, split, normal, and defect folder names.
- Inspect examples from each label to ensure that folder names match the visible content.
- Confirm that supported image files decode and that no unexpected files are treated as images.
- Record dataset provenance and a manifest of filenames/counts without relying only on notebook output.
- Confirm Python, PyTorch, TorchVision, CUDA, and DINOv2 checkpoint details.
- Confirm available GPU memory and choose only a compatible backbone/input/batch configuration.

During training:

- Save the effective configuration with the run output.
- Save the exact split assignments so later comparisons use the same images.
- Record per-epoch training and validation metrics; do not treat training loss alone as evidence of generalization.
- Capture skipped categories, warnings, and exceptions rather than silently omitting them.

After training:

- Verify that each test set contains both normal and anomalous labels before interpreting AUROC.
- Save numeric I-AUROC and, only when valid masks exist, P-AUROC values in the same results artifact.
- Inspect the CSV and confirm that its category counts agree with the split manifest.
- Keep attention overlays qualitative until validated against masks.
- Record limitations, threshold selection, and any deviations from the original configuration.

B.4 Result Reporting Template

A future experiment should report one row per category and include at least the fields in Table B.2. Blank values should remain blank or be marked unavailable; they should not be inferred from figures.

Table B.2: Recommended fields for future category-level results.

Field	Reporting instruction
Dataset/version	State the source, version, date, and any filtering
Split protocol	State train/validation/test sources, seed, and fixed assignments
Category and defect groups	Report category and the counts of each subgroup
Training counts	Give normal and anomalous numbers separately
Test counts	Give normal and anomalous numbers separately
I-AUROC	Report score and, where possible, uncertainty across repeated runs
Threshold metrics	Report threshold selection and confusion matrix on a held-out test set
P-AUROC	Include only when pixel masks and pixel scores are valid and aligned
Compute setup	State hardware, software versions, time, and peak memory if measured
Artifacts	Identify configuration, split manifest, result file, and visualization outputs

Appendix C

Glossary and Notation

Table C.1: Terms used in this report.

Term	Meaning in this project
Anomaly	An image labeled as non-normal; defect folders are combined into this binary class
AUROC	Area under the receiver operating characteristic curve; here used to assess image-level score ranking
Backbone	Pretrained DINOv2-Large model used to extract image patch features
Cross-attention	Operation using the fused image descriptor as query and graph tokens as keys and values
DINOv2	Self-supervised visual feature model used as a frozen backbone
Feature depth	A transformer-block output; four final depths are extracted in this configuration
Graph node	One patch token in the 16×16 spatial grid
Graph mask	Symmetric Boolean adjacency defining which patch pairs are allowed to attend
I-AUROC	Image-level AUROC calculated from image anomaly probabilities
Knowledge loss	Reconstruction MSE between decoded fused features and detached raw pooled features
MANTA	Name used by the supplied notebook for the graph-architecture agriculture anomaly pipeline
Multi-depth features	Feature maps extracted at multiple transformer depths, sharing the same spatial grid
Normal	Images located in the detected normal folder and assigned label 0
P-AUROC	Pixel-level AUROC; code is present but no verifiable numeric result is saved in the supplied output
Accuracy	Fraction of all images correctly classified at a specified threshold
Confusion matrix	Counts of true negatives, false positives, false negatives, and true positives
F1-score	Harmonic mean of anomaly-class precision and recall
Precision	Fraction of anomaly predictions that are true anomalies

Term	Meaning in this project
Recall	Fraction of true anomalies detected by the classifier
Patch token	Vector representation associated with one image patch
Seed	Fixed random state value used to support repeatable shuffling and initialization
Supervised contrastive loss	Batch-based embedding loss using class labels to define positive pairs

Notation

Symbol	Definition
B	Batch size
N	Number of patch tokens, 256 for a 16×16 grid
D	Projected token width, 256 in the saved configuration
A_{ij}	Boolean graph adjacency between patch positions i and j
$Z_{t,i}$	Projected pre-graph token at position i
H_i	Graph-refined token at position i
z	Mean-pooled projected feature vector
h	Mean-pooled graph-refined feature vector
g	Learned sigmoid fusion gate
f	Gated combination of z and h
y	Binary image label: 0 normal, 1 anomalous
τ	Temperature parameter 0.1 for supervised contrastive loss

Bibliography

[1] K. Roth, L. Pemula, J. Zepeda, B. Sch"olkopf, T. Brox, and P. Gehler, "Towards Total Recall in Industrial Anomaly Detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 14318–14328.

[2] T. Defard, A. Setkov, A. Loesch, and R. Audigier, "PaDiM: A Patch Distribution Modeling Framework for Anomaly Detection and Localization," in *Proceedings of the International Conference on Pattern Recognition (ICPR)*, 2021, pp. 475–489.

[3] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger, "MVTec AD – A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 9592–9600.

[4] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *International Conference on Learning Representations*, 2021.

[5] M. Oquab, T. Darcet, T. Moutakanni, et al., "DINOv2: Learning Robust Visual Features without Supervision," arXiv:2304.07193, 2023.

[6] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph Attention Networks," in *International Conference on Learning Representations (ICLR)*, 2018.

[7] P. Khosla, P. Teterwak, C. Wang, et al., "Supervised Contrastive Learning," in *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[8] J. A. Hanley and B. J. McNeil, "The Meaning and Use of the Area under a Receiver Operating Characteristic (ROC) Curve," *Radiology*, vol. 143, no. 1, pp. 29–36, 1982.

[9] L. Fan, D. Fan, Z. Hu, Y. Ding, D. Di, K. Yi, M. Pagnucco, and Y. Song, "MANTA: A Large-Scale Multi-View and Visual-Text Anomaly Detection Dataset for Tiny Objects," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025, pp. 25518–25527.

[10] A. Vaswani, N. Shazeer, N. Parmar, et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[11] M. Caron, H. Touvron, I. Misra, et al., "Emerging Properties in Self-Supervised Vision Transformers," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[12] J. Zhou, C. Wei, H. Wang, et al., "iBOT: Image BERT Pre-Training with Online Tokenizer," in *International Conference on Learning Representations (ICLR)*, 2022.

[13] L. Ruff, R. A. Vandermeulen, N. Goernitz, et al., "Deep One-Class Classification," in *Proceedings of the 35th International Conference on Machine Learning*, vol. 80, 2018.

[14] C. Li, K. Sohn, J. Yoon, and T. Pfister, "CutPaste: Self-Supervised Learning for Anomaly Detection and Localization," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[15] V. Zavrtanik, M. Kristan, and D. Skočaj, "DRAEM—A Discriminatively Trained Reconstruction Embedding for Surface Anomaly Detection," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[16] G. Wang, S. Han, E. Ding, and D. Huang, "Student-Teacher Feature Pyramid Matching for Anomaly Detection," in *Proceedings of the British Machine Vision Conference (BMVC)*, 2021.

[17] J. Yu, Y. Zheng, X. Wang, W. Li, Y. Wu, R. Zhao, and L. Wu, "FastFlow: Unsupervised Anomaly Detection and Localization via 2D Normalizing Flows," arXiv:2111.07677, 2021.

[18] H. Deng and X. Li, "Anomaly Detection via Reverse Distillation from One-Class Embedding," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 9737–9746.

[19] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A Simple Framework for Contrastive Learning of Visual Representations," in *Proceedings of the 37th International Conference on Machine Learning*, 2020.

[20] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," in *International Conference on Learning Representations (ICLR)*, 2017.

[21] C. Ying, T. Cai, S. Luo, et al., "Do Transformers Really Perform Badly for Graph Representation?" in *Advances in Neural Information Processing Systems*, vol. 34, 2021.

[22] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *International Conference on Learning Representations (ICLR)*, 2019.

[23] D. Hendrycks and K. Gimpel, "Gaussian Error Linear Units (GELUs)," arXiv:1606.08415, 2016.

[24] P. Bergmann, K. Batzner, M. Fauser, D. Sattlegger, and C. Steger, "Beyond Dents and Scratches: Logical Constraints in Unsupervised Anomaly Detection and Localization," *International Journal of Computer Vision*, vol. 130, pp. 947–969, 2022.

[25] Y. Zou, J. Jeong, L. Pemula, D. Zhang, and O. Dabeer, "SPot-the-Difference Self-supervised Pre-training for Anomaly Detection and Segmentation," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022, pp. 392–408.

[26] K. Batzner, L. Heckler, and R. König, "EfficientAD: Accurate Visual Anomaly Detection at Millisecond-Level Latencies," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2024, pp. 128–138.

[27] J. Jeong, Y. Zou, T. Kim, D. Zhang, A. Ravichandran, and O. Dabeer, "WinCLIP: Zero-/Few-Shot Anomaly Classification and Segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 19606–19616.

[28] Q. Zhou, G. Pang, Y. Tian, S. He, and J. Chen, "AnomalyCLIP: Object-agnostic Prompt Learning for Zero-shot Anomaly Detection," in *International Conference on Learning Representations (ICLR)*, 2024.

[29] Z. Liu, Y. Zhou, Y. Xu, and Z. Wang, "SimpleNet: A Simple Network for Image Anomaly Detection and Localization," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 20402–20411.